

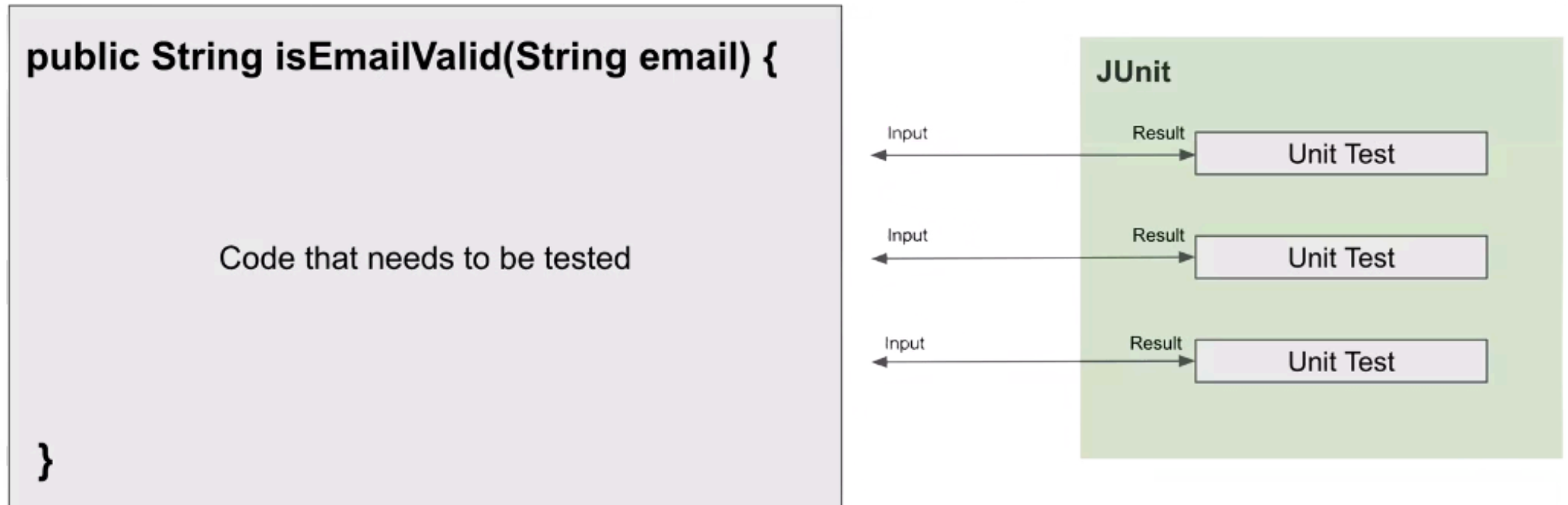


# What Is a Unit Test?

Understanding the fundamentals of unit testing  
in Java development

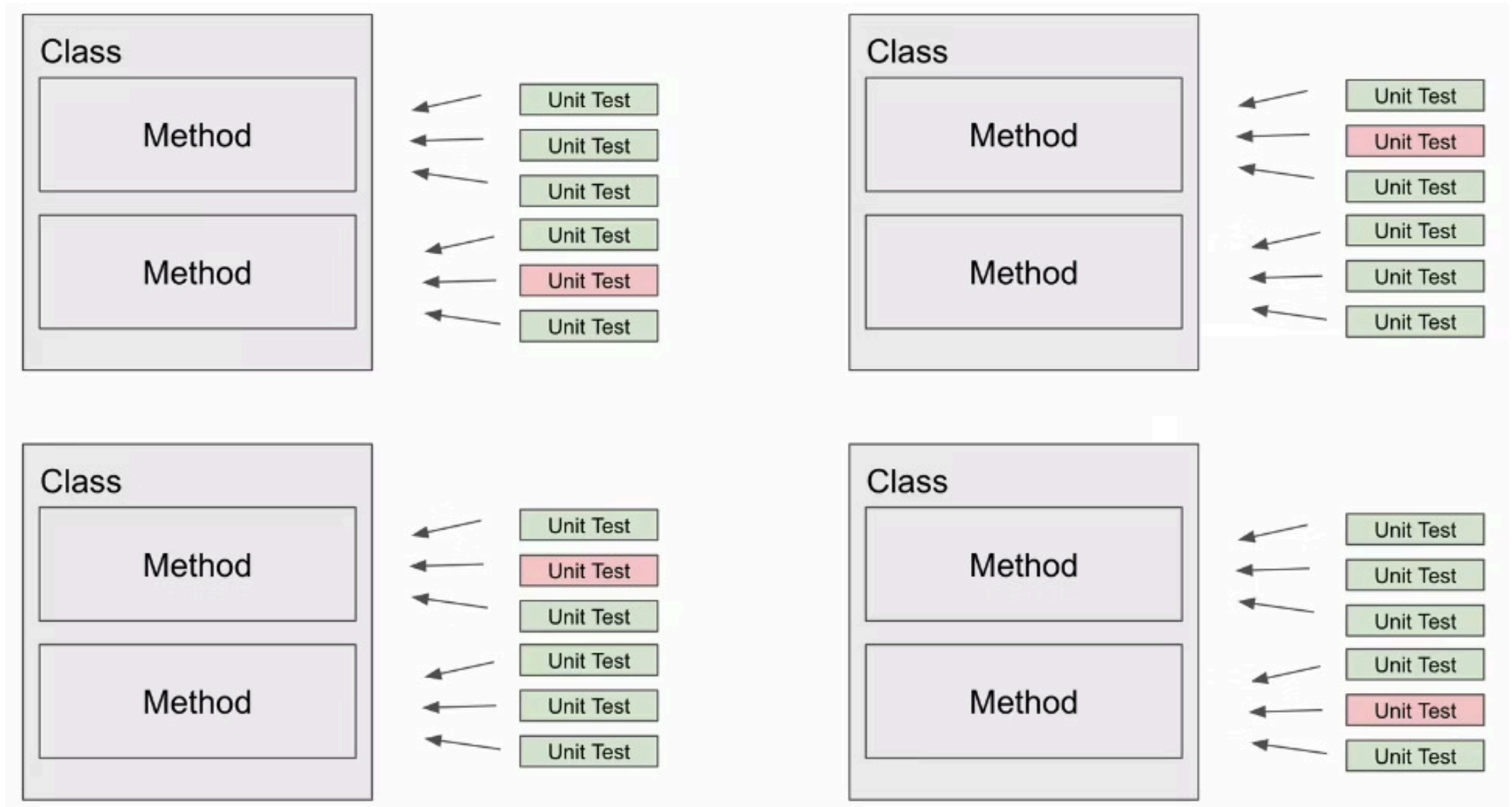
# What Is a Unit Test?

- A **unit test** is a **small, self-contained** Java **method** that **tests one** specific **part** of your **code**



**Source:** Testing Java: JUnit 5, Mockito, Testcontainers, REST Assured by Sergey Kargopolov.

# Red vs. Green



**Source:** Testing Java: JUnit 5, Mockito, Testcontainers, REST Assured by Sergey Kargopolov.

# Structure of a Unit Test

A **typical unit test** follows three **stages**:



## Arrange

create or prepare objects  
needed for the test



## Act

invoke the method under test  
with specific inputs



## Assert

verify that the actual result  
matches the expected value

- **JUnit** provides the API for writing **assertions** such as `assertEquals()`
- Each **test passes or fails** independently
- **Passing tests** appear in **green**; **failing tests** appear in **red**



# Unit Test Example

```
public class Calculator {  
  
    public int integerDivision(int dividend, int divisor) {  
        return dividend / divisor;  
    }  
  
}
```

```
@Test  
void testIntegerDivision_whenValidValuesProvided_shouldReturnExpectedResult() {  
  
    //Arrange  
    Calculator calculator = new Calculator();  
  
    //Act  
    int result = calculator.integerDivision(4,2);  
  
    //Assert  
    assertEquals(2, result, "4/2 should have returned 2");  
  
}
```

# Unit Test Example: Red vs. Green in IntelliJ

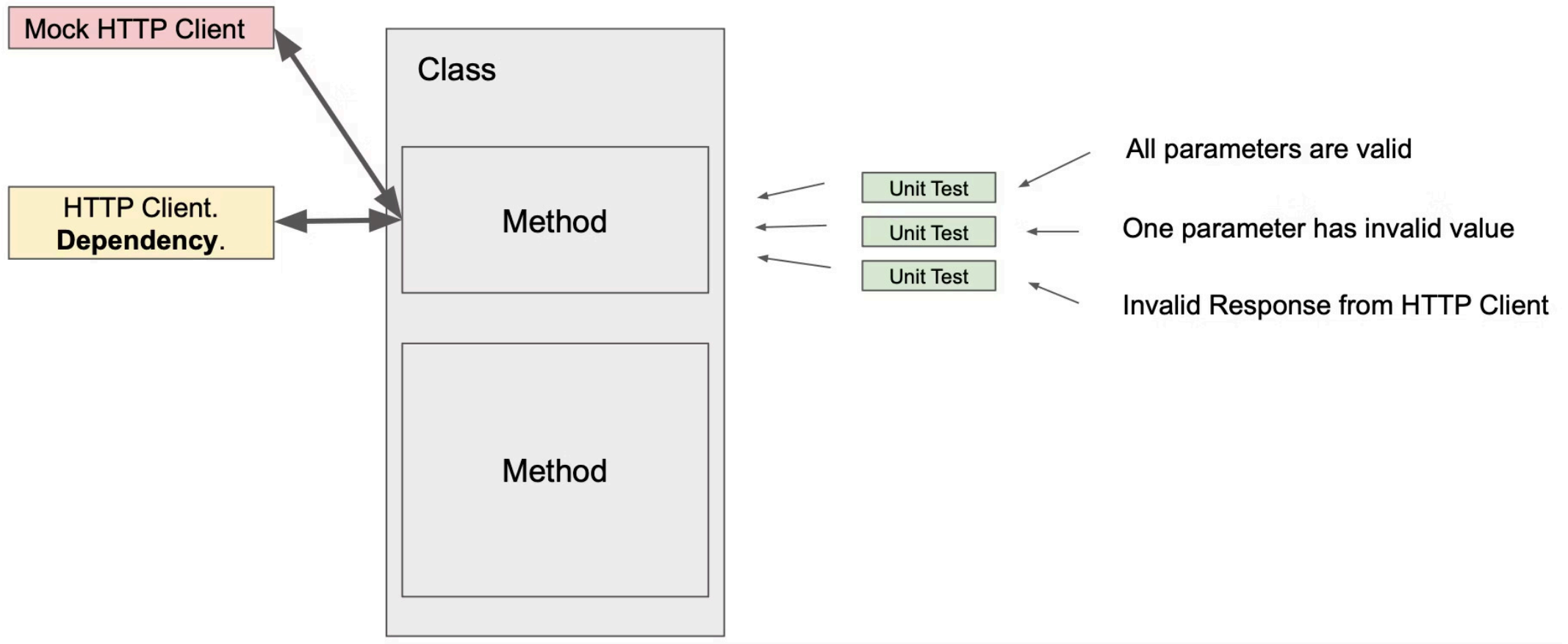
```
12      @Test
13      void integerDivision() {
14          // Arrange
15          Calculator calculator = new Calculator();
16
17          // Act
18          int actualResult = calculator.integerDivision( dividend: 4, divisor: 2);
19
20          // Assert
21          assertEquals( expected: 2, actualResult);
22      }
```



```
12      @Test
13      void integerDivision() {
14          // Arrange
15          Calculator calculator = new Calculator();
16
17          // Act
18          int actualResult = calculator.integerDivision( dividend: 4, divisor: 2);
19
20          // Assert
21          assertEquals( expected: 2, actualResult);
22      }
```

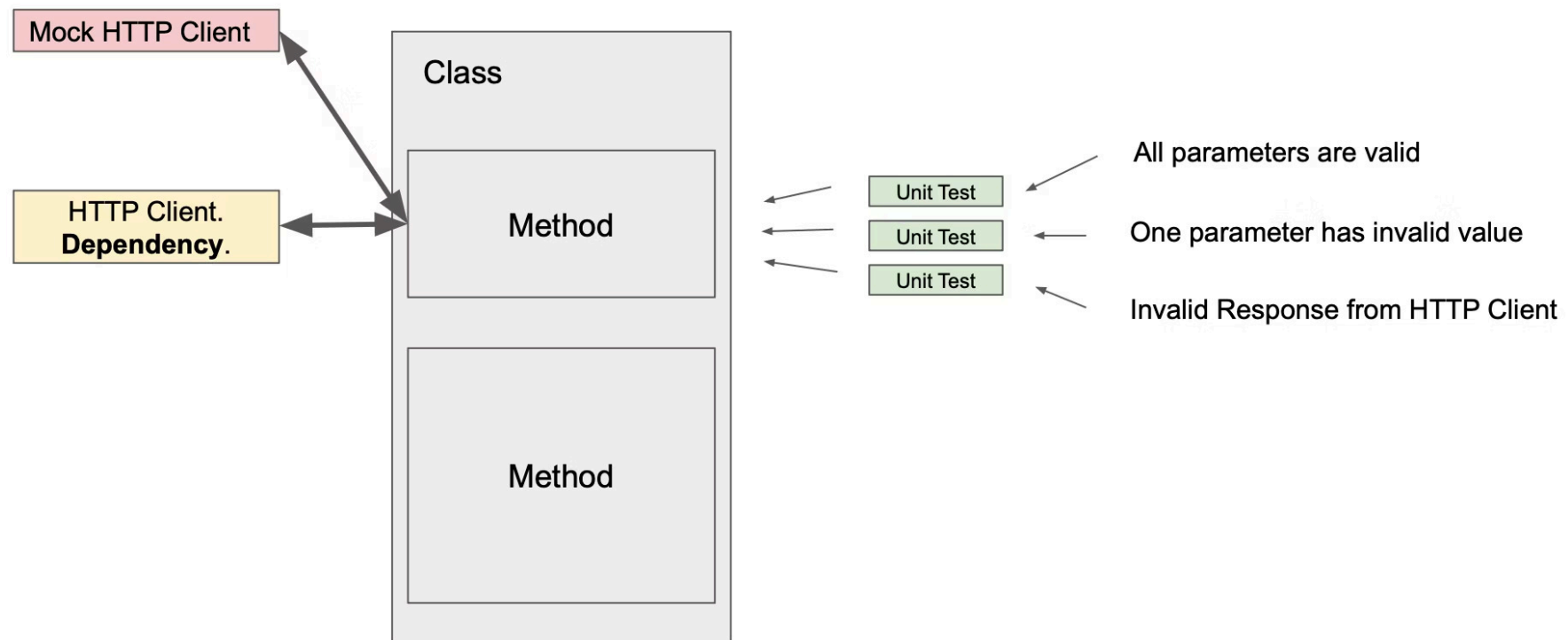


# Why Unit Tests Are Fast and Reliable



# Why Unit Tests Are Fast and Reliable (contd.)

- **Unit tests run quickly** because external **dependencies** are **replaced** with **fakes** or **mocks**
  - Examples include **fake objects**, **mock objects**, and **spy objects**
- **Mocking** prevents real external operations (e.g., HTTP calls) from executing
- This isolation ensures the **test evaluates *only*** the **internal** Java **logic** of the **method**
- **Developers** often write **several tests** to **check behavior** under **valid**, **invalid**, and **edge-case conditions**





# Conclusion

- **Unit tests** validate **small**, isolated parts of your application
- They use **JUnit's simple API** and follow a predictable **Arrange-Act-Assert** pattern
- **Mocks** help keep tests fast, focused, and reliable
- Writing **multiple tests per method** increases confidence in code quality

# Open Floor Discussion

**Which part of unit testing do you find most challenging: Arrange, Act, or Assert?**

**Have you used mocks before in Java development?**

**Do you prefer testing simple utility methods or more complex business logic first?**

